



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251210
Nama Lengkap	Gabriel Ekklesia
Minggu ke / Materi	13 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1

Pengertian Rekursif

Fungsi rekursif merupakan salah satu konsep penting dalam pemrograman, yaitu fungsi yang dapat memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan. Konsep ini sering digunakan pada kasus yang memiliki pola berulang atau terstruktur, khususnya dalam bentuk perhitungan matematis. Dengan menggunakan rekursif, suatu masalah besar dapat dipecah menjadi bagian-bagian yang lebih kecil namun tetap memiliki pola yang sama. Meskipun demikian, penggunaan fungsi rekursif harus dilakukan dengan hati-hati karena jika tidak dirancang dengan benar, fungsi ini dapat berjalan tanpa henti (infinite loop) sehingga berpotensi menghabiskan memori dan menyebabkan program mengalami hang atau error.

Agar fungsi rekursif dapat berjalan dengan baik dan tidak menimbulkan masalah, diperlukan adanya kondisi penghentian yang jelas. Fungsi rekursif akan terus melakukan pemanggilan terhadap dirinya sendiri sampai kondisi berhenti tersebut terpenuhi. Oleh karena itu, dalam sebuah fungsi rekursif terdapat dua komponen utama yang sangat penting, yaitu base case dan recursive case.

- Base case merupakan kondisi yang menjadi titik berhenti dari proses rekursif, sehingga ketika kondisi ini tercapai, fungsi tidak lagi memanggil dirinya sendiri.
- recursive case adalah bagian yang berisi proses pemanggilan fungsi itu sendiri secara berulang, biasanya dengan parameter yang mendekati kondisi base case.

Dengan adanya kedua komponen ini, fungsi rekursif dapat berjalan secara terarah, terkontrol, dan mampu menghasilkan solusi yang benar tanpa menimbulkan perulangan tak terbatas.

MATERI 2

Kelebihan dan Kekurangan

Keunggulan Fungsi Rekursif:

1. Penulisan kode lebih singkat, rapi, dan terlihat lebih elegan dibandingkan metode biasa (iteratif).
2. Memudahkan penyelesaian masalah yang kompleks dengan cara memecahnya menjadi sub-masalah yang lebih kecil dan terstruktur.
3. Sangat cocok digunakan untuk permasalahan yang memiliki pola berulang, seperti perhitungan matematis, tree, dan graph.
4. Membantu membuat logika program menjadi lebih sistematis dan mudah dipahami jika konsepnya sudah dikuasai.

Kelemahan Fungsi Rekursif:

1. Membutuhkan memori lebih besar karena setiap pemanggilan fungsi akan disimpan dalam *call stack*.
2. Kurang efisien dalam hal waktu eksekusi karena adanya pemanggilan fungsi yang berulang-ulang.
3. Proses debugging lebih sulit karena alur program tidak berjalan secara linear.
4. Terkadang sulit dipahami, terutama bagi pemula yang belum terbiasa dengan konsep rekursif.
5. Berisiko menyebabkan *infinite loop* jika tidak memiliki *base case* yang jelas, sehingga dapat membuat program hang atau error.

MATERI 3

Bentuk Umum dan Studi Kasus

Bentuk umum fungsi rekursif pada Python:

```
def function_name(parameter_list):  
    function_name(...)
```

Sebenarnya semua fungsi rekursif pasti memiliki solusi iteratifnya. Misalnya pada contoh kasus faktorial berikut: Faktorial adalah menghitung perkalian deret angka $1 \times 2 \times 3 \times \dots \times n$. Algoritma untuk menghitung faktorial adalah:

1. Tanyakan n
2. Siapkan variabel total untuk menampung hasil perkalian faktorial dan set nilai awal dengan 0
3. Loop dari $i = 1$ hingga n untuk mengerjakan:
4. $\text{total} = \text{total} * i$
5. Tampilkan total

faktorial dapat dihitung dengan rumus:

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{if } n > 0 \end{cases}$$

Pseudocode rekursif factorial:

Pseudocode (recursive):

function factorial is:

input: integer n such that $n \geq 0$

output: $[n \times (n-1) \times (n-2) \times \dots \times 1]$

1. if n is 0, return 1
2. otherwise, return $[n \times \text{factorial}(n-1)]$

end factorial

Contoh program dapat kita lihat pada code dibawah ini:

```
def faktorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return faktorial(n-1) * n

print(faktorial(4))
```

24

Cara perhitungan:

```
1. calc_factorial(4)           # 1st call with 4
2. 4 * calc_factorial(3)       # 2nd call with 3
3. 4 * 3 * calc_factorial(2)   # 3rd call with 2
4. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1
5. 4 * 3 * 2 * 1               # return from 4th call as number=1
6. 4 * 3 * 2                   # return from 3rd call
7. 4 * 3 * 2                   # return from 2nd call
8. 4 * 6                       # return from 1st call
9. 24
```

Contoh lain:

Di sebuah kelas informatika, guru memberikan tugas kepada siswa untuk menghitung jumlah susunan kemungkinan dari beberapa benda.

Suatu hari, Andi memiliki 4 buah buku berbeda yang ingin ia susun di rak. Banyaknya cara menyusun buku tersebut dapat dihitung menggunakan konsep faktorial.

Diketahui program Python berikut digunakan untuk menghitungnya:

```
def faktorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return faktorial(n-1) * n

nilai = int(input("Masukkan nilai: "))
print("Hasil faktorial =", faktorial(nilai))
```

```
Masukkan nilai: 5
Hasil faktorial = 120
```

```
1 faktorial(5)
2 = 5 × faktorial(4)
3 = 5 × 4 × faktorial(3)
4 = 5 × 4 × 3 × faktorial(2)
5 = 5 × 4 × 3 × 2 × faktorial(1)
6 = 5 × 4 × 3 × 2 × 1
7 = 120
```

Contoh lainnya:

Di sebuah acara kelulusan, terdapat 5 siswa yang akan naik ke panggung untuk menerima penghargaan. Mereka harus berdiri dalam satu barisan untuk difoto oleh panitia.

Karena posisi berdiri bisa berbeda-beda, panitia ingin mengetahui berapa banyak susunan posisi yang mungkin dari kelima siswa tersebut.

Code:

```
def faktorial(n):
    hasil = 1
    for i in range(1, n + 1):
        hasil *= i
    return hasil

nilai = int(input("Masukkan nilai: "))

print("Hasil faktorial =", faktorial(nilai))
```

```
Masukkan nilai: 4
Hasil faktorial = 24
```

Cara perhitungan:

```
1 faktorial(4)
2 = 4 × 3 × 2 × 1
3 = 12 × 2 × 1
4 = 24 × 1
5 = 24
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

```
def cek_bilangan_prima(x, y):  
    if x <= 1:  
        return False  
  
    if y == 1:  
        return True  
  
    if x % y == 0:  
        return False  
  
    return cek_bilangan_prima(x, y - 1)  
  
angka = int(input("Masukkan bilangan: "))  
  
if cek_bilangan_prima(angka, angka - 1):  
    print(angka, "adalah bilangan PRIMA")  
else:  
    print(angka, "bukan bilangan PRIMA")
```

Pertama kita membuat fungsi `cek_bilangan_prima(x, y)` untuk mengecek bilangan prima secara rekursif, di mana `x` adalah angka dan `y` adalah pembagi.

Kemudian jika `x <= 1`, langsung dikembalikan `False` karena bukan bilangan prima.

Selanjutnya jika `y == 1`, dikembalikan `True` karena tidak ada pembagi lain.

Lalu jika `x % y == 0`, dikembalikan `False` karena ada pembagi selain 1 dan dirinya sendiri.

Jika tidak, fungsi dipanggil kembali dengan `y - 1`.

Terakhir, program meminta input user, lalu mengecek hasilnya dan menampilkan apakah bilangan tersebut prima atau bukan.

SOAL 2

```
def palindrom(teks):
    if len(teks) <= 1:
        return True

    kata_depan = teks[0]
    kata_belakang = teks[-1]

    if kata_depan == kata_belakang:
        return palindrom(teks[1:-1])
    else:
        return False

kalimat = input("Masukkan kalimat: ")

kalimat_baru = kalimat.lower()
kalimat_baru = kalimat_baru.replace(" ", "")
hasil = palindrom(kalimat_baru)

if hasil:
    print("Kalimat tersebut adalah PALINDROM")
else:
    print("Kalimat tersebut BUKAN PALINDROM")
```

Pertama kita membuat fungsi `palindrom(teks)` untuk mengecek apakah suatu kalimat termasuk palindrom secara rekursif.

Kemudian jika `len(teks) <= 1`, fungsi langsung mengembalikan `True` karena jika tinggal 1 huruf atau kosong pasti palindrom.

Selanjutnya kita ambil huruf depan dengan `teks[0]` dan huruf belakang dengan `teks[-1]`.

Lalu kita cek jika huruf depan sama dengan huruf belakang, maka fungsi dipanggil kembali dengan `teks[1:-1]` (menghilangkan huruf depan dan belakang).

Jika tidak sama, langsung dikembalikan `False` karena bukan palindrom.

Setelah itu program meminta input user, lalu diubah menjadi huruf kecil dengan `lower()` dan menghapus spasi dengan `replace()`.

Terakhir, hasil dicek dan ditampilkan apakah kalimat tersebut palindrom atau bukan.

SOAL 3

```
def jumlah_ganjil(x, i=1):  
    if (2 * i - 1) > x:  
        return 0  
  
    return (2 * i - 1) + jumlah_ganjil(x, i + 1)  
  
angka = int(input("Masukkan batas bilangan ganjil: "))  
  
hasil = jumlah_ganjil(angka)  
print("Jumlah deret ganjil =", hasil)
```

Pertama kita membuat fungsi `jumlah_ganjil(x, i=1)` untuk menghitung jumlah deret bilangan ganjil secara rekursif sampai batas yang dimasukkan user.

Kemudian kita cek kondisi `if (2 * i - 1) > x`, artinya jika bilangan ganjil yang sedang dihitung sudah lebih besar dari batas `x`, maka fungsi akan mengembalikan nilai 0 sebagai kondisi berhenti.

Selanjutnya jika masih belum melewati batas, maka program akan menjumlahkan bilangan ganjil saat ini yaitu $(2 * i - 1)$ dengan hasil pemanggilan fungsi berikutnya `jumlah_ganjil(x, i + 1)`.

Setelah itu program meminta input dari user berupa batas bilangan ganjil, lalu disimpan dalam variabel `angka`.

Terakhir fungsi dipanggil dengan `jumlah_ganjil(angka)` dan hasilnya ditampilkan sebagai jumlah seluruh deret bilangan ganjil sampai batas tersebut.

SOAL 4

```
def jumlah_digit(x):  
    if x < 10:  
        return x  
    else:  
        return (x % 10) + jumlah_digit(x // 10)  
  
angka = int(input("Masukkan angka: "))  
print("Jumlah digit:", jumlah_digit(angka))
```

Pertama kita membuat fungsi `jumlah_digit(x)` untuk menghitung jumlah digit dari sebuah bilangan secara rekursif.

Kemudian jika $x < 10$, fungsi langsung mengembalikan nilai x karena angka satu digit sudah tidak bisa dipecah lagi.

Selanjutnya jika x lebih besar atau sama dengan 10, maka program akan memisahkan digit terakhir dengan $x \% 10$ dan menjumlahkannya dengan hasil pemanggilan fungsi `jumlah_digit(x // 10)` yang berarti angka dibagi 10 untuk menghilangkan digit terakhir.

Proses ini akan terus berulang sampai tersisa satu digit angka saja.

Setelah itu program meminta input dari user berupa sebuah angka yang disimpan dalam variabel `angka`.

Terakhir, fungsi dipanggil dengan `jumlah_digit(angka)` dan hasil penjumlahan semua digit ditampilkan ke layar.

SOAL 5

```
def kombinasi(x, y):  
    if y == 0 or y == x:  
        return 1  
    elif y > x or y < 0:  
        return 0  
    else:  
        return kombinasi(x-1, y-1) + kombinasi(x-1, y)  
  
x = int(input("Masukkan nilai x: "))  
y = int(input("Masukkan nilai y: "))  
  
print("Hasil kombinasi C(", x, ", ", y, ") =", kombinasi(x, y))
```

Pertama kita membuat fungsi kombinasi(x, y) untuk menghitung nilai kombinasi (C) secara rekursif, yaitu cara memilih y elemen dari x elemen tanpa memperhatikan urutan.

Kemudian pada kondisi dasar, jika $y == 0$ atau $y == x$, maka fungsi langsung mengembalikan nilai 1 karena hanya ada satu cara memilih (atau tidak memilih sama sekali / memilih semuanya).

Selanjutnya jika $y > x$ atau $y < 0$, maka fungsi mengembalikan 0 karena kondisi tersebut tidak valid dalam perhitungan kombinasi.

Jika tidak memenuhi kondisi dasar tersebut, maka fungsi akan menghitung kombinasi dengan rumus rekursif yaitu $kombinasi(x-1, y-1) + kombinasi(x-1, y)$. Ini berarti kombinasi dipecah menjadi dua bagian: memilih elemen saat ini dan tidak memilih elemen saat ini.

Setelah itu program meminta input dari user berupa nilai x dan y.

Terakhir fungsi dipanggil dengan kombinasi(x, y) dan hasilnya ditampilkan sebagai nilai kombinasi C(x, y).

LINK GITHUB: https://github.com/gilbetjuliesa-crypto/71251210_Gabriel_Ekklesia.git